



Module

Système d'Exploitation et Programmation Système

Chapitre 3

Processus & Threads (1/2)

Pr. MANALE Boughanja

m.boughanja@ump.ac.ma

Terminologie

- Un **programme** est un fichier source écrit dans un langage donnée, par exemple un programme C, C++, ou Java
- Un programme désigne également le fichier en langage binaire obtenu après compilation, on parle également de **programme exécutable**.
- Un **processus** est un programme en **cours exécution** par le système d'exploitation.
- Un processus peuvent être **décomposés** en unités plus petites et plus légères appelées **threads**. Alors qu'un processus représente un programme en exécution, un **thread** est **l'unité d'exécution** la plus petite à **l'intérieur** d'un processus.

Définition

Un processus est constitué d'un programme binaire : instructions + données statiques, associé à un contexte d'exécution :

- Pile + tas
- Données dans les registres du processeur
- Instruction courante

Exemple: Processus de Comptage

N.B:

- ✓ La **Pile** est utilisée pour se souvenir où il doit revenir après avoir fait une tâche
- ✓ Le **Tas** est utilisé pour ranger les nouvelles choses créées pendant l'exécution.

Programme Binaire	Instructions : Compter 1, puis 2, puis 3, etc. Données Statiques : Le nombre maximum est 5.
Pile + Tas	Stocke temporairement les nombres qui doivent être affichés.
Registres du Processeur	Contient le nombre actuel que l'on est en train de traiter (par exemple, le nombre 3).
Instruction Courante	L'ordinateur est à l'étape : "Afficher le nombre actuel".

Chargement du programme en mémoire

6

Objectif: Placer le programme exécutable, stocké sur le disque, dans la mémoire physique pour l'exécuter. Le chargeur de l'OS doit également ajouter des segments pour gérer entre autre la pile et le tas.

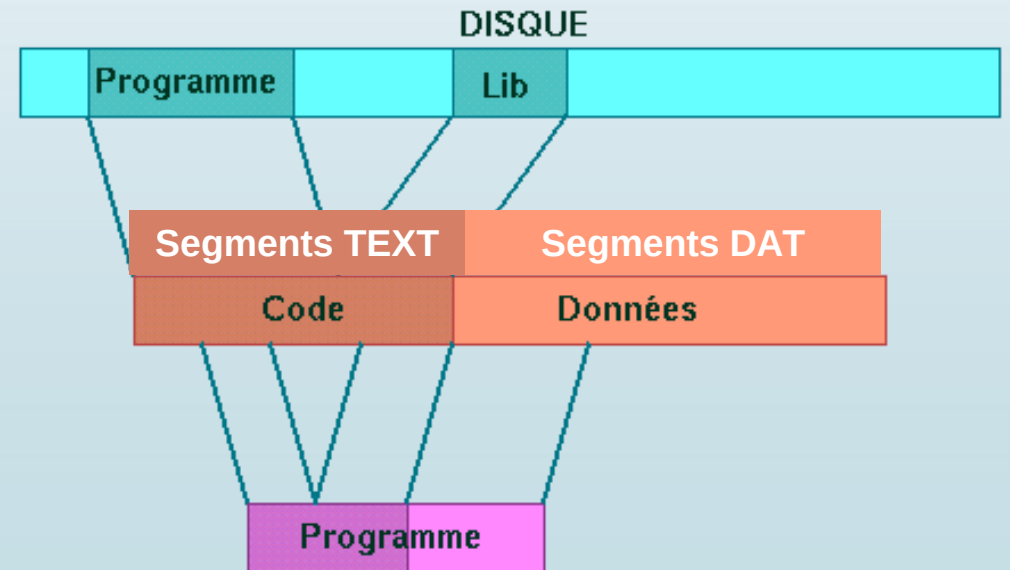
Définition: La mémoire réelle, ou physique, correspond à la quantité de mémoire installée sur le système. Elle peut être vue comme un tableau d'octets.

Chargement du programme en mémoire

6

Objectif: Placer le programme exécutable, stocké sur le disque, dans la mémoire physique pour l'exécuter. Le chargeur de l'OS doit également ajouter des segments pour gérer entre autre la pile et le tas.

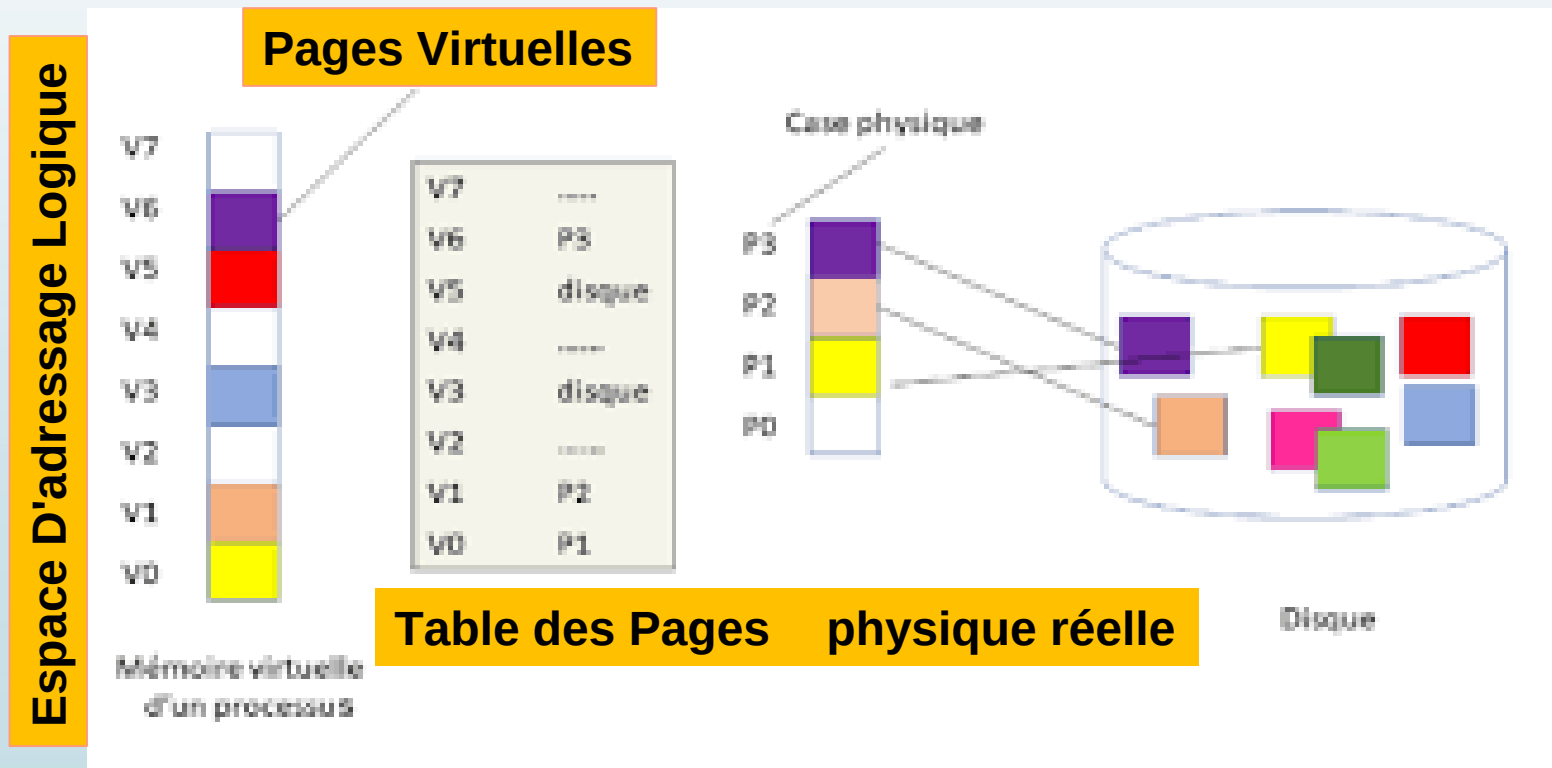
Définition: La mémoire réelle, ou physique, correspond à la quantité de mémoire installée sur le système. Elle peut être vue comme un tableau d'octets.



Mémoire virtuelle

Comment l'OS gère plusieurs processus sans qu'ils s'écrasent ?

La **Mémoire Virtuelle** est une technique utilisée par le Système d'Exploitation (OS) qui permet à chaque processus d'avoir l'impression d'utiliser un **espace d'adressage mémoire grand, contigu et privé**



Processus de chargement

Diviser le programme en pages

Pour que l'OS puisse gérer la mémoire, il découpe le programme binaire (le code et les données) en petits morceaux de taille fixe. Ces morceaux sont appelés **Pages Virtuelles**.

Passage d'un adressage linéaire à un adressage en mémoire virtuelle.

traduit ces adresses en **Adresses Virtuelles**, spécifiques à l'espace mémoire **privé** de ce processus.

Vérification de la disponibilité de cases en mémoire réelle.

Vérification des espaces libres dans la RAM s'appellent des **Cases Physiques** (*Frames*).

Chargement des pages utilisées dans les cases disponibles

L'OS ne charge **pas** tout le programme d'un coup. Il ne copie du disque vers la RAM que les pages dont le processus a **immédiatement besoin** pour commencer.

Exemple de passage adresse linéaire-adresse virtuel

L'Hypothèse: Le programme est construit comme s'il commençait à l'adresse zéro

Exemple : Le code veut lire le compteur qui se trouve à 100 octets du début du segment DATA.

La Décision de l'OS (Adresse Virtuelle de Base) : C'est l'**Adresse Virtuelle de Base (AVB)**.

L'OS décide que le segment DATA commencera à l'adresse **0x400000** (l'AVB)

Nouvelle Adresse Virtuelle = Adresse Virtuelle de Base + Décalage Linéaire

Convertir le 100,décimal en hexadécimal : 100 base 10 = 64 base 16

Calcule: $0x400000 + 0x00064 = 0x400064$

Gestion des processus par le système

Information:

- **ps**: Liste des processus propriétaire de chaque processus, pid et parent pid des processus.

Utilisation:

- -e : tous les processus
- axjf : arborescence des processus
- -u username : processus d'un utilisateur

Plus d'options avec man ps !

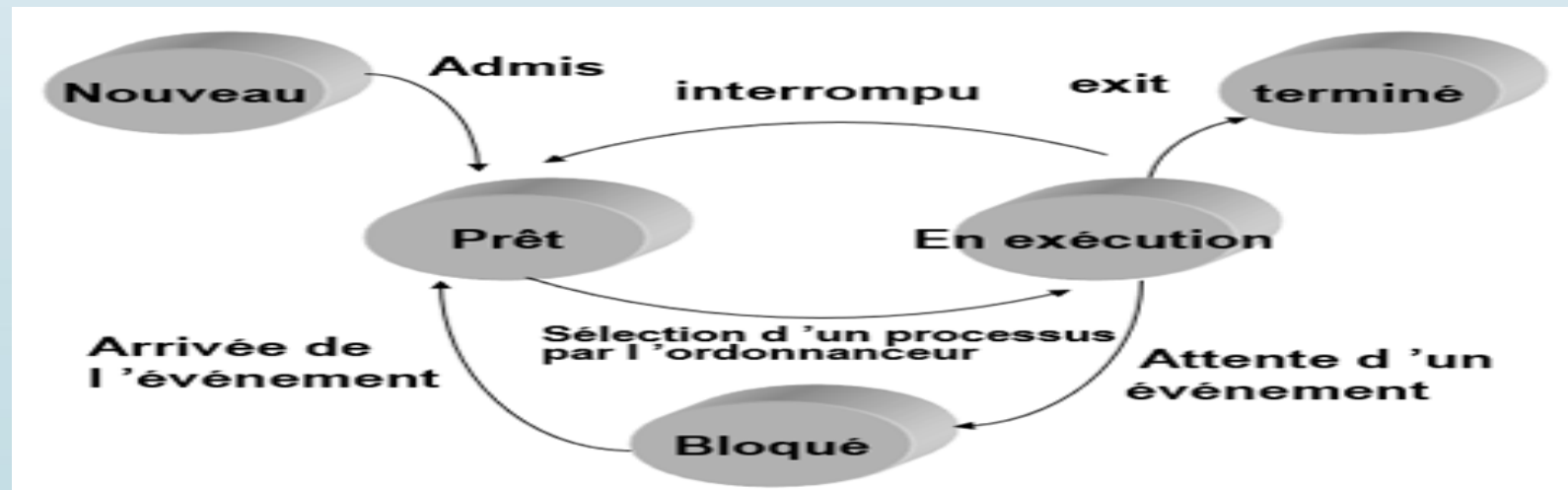
Etat Processus

12

Un processus peut prendre un de ces 3 états :

- **En cours(Running)** d'exécution le programme utilise le processeur.
- **Prêt (Ready)** exécutable, temporairement arrêté pour laisser un autre processus.
- **Bloqué (Sleep)** ne peut pas s'exécuter tant qu'un évènement externe ne se produit pas.

Le passage de En cours à Prêt et inversement est géré par l'ordonnanceur de processus.



Ordonnancement des Processus

L'**ordonnanceur** est un composant clé du système d'exploitation chargé de **décider** quel processus doit s'exécuter à un **moment donné**. Son rôle est crucial pour assurer une **gestion** efficace du processeur. Le système d'exploitation associe pour chaque processus un bloc de contrôle appelé en anglais Process Control Bloc et abrégé PCB qui contient les informations nécessaires sur un processus.

Identificateur de processus

Un numéro unique attribué à chaque processus.

Etat du processus

Indique l'état actuel du processus (par exemple, En cours, Prêt, Bloqué).

Compteur ordinal

Pointeur vers la prochaine instruction que le processus doit exécuter.

Contexte (Registre, pointeur pile..)

Stocke les valeurs des registres lorsque le processus n'est pas en cours d'exécution.

Informations sur l'ordonnancement

Détails sur l'utilisation de la mémoire du processus

Informations mémoire

Détails sur les périphériques d'entrée/sortie utilisés

Informations sur les E/S

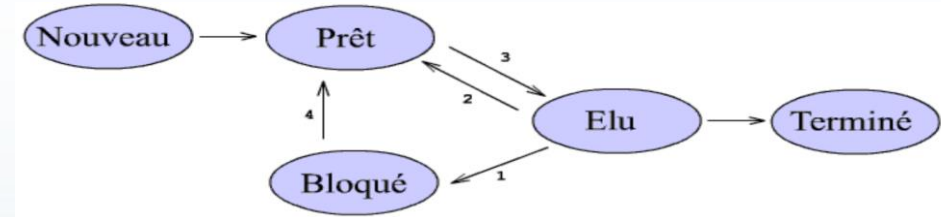
Traque l'utilisation du CPU, la priorité du processus

Les niveaux d'ordonnancement des processus



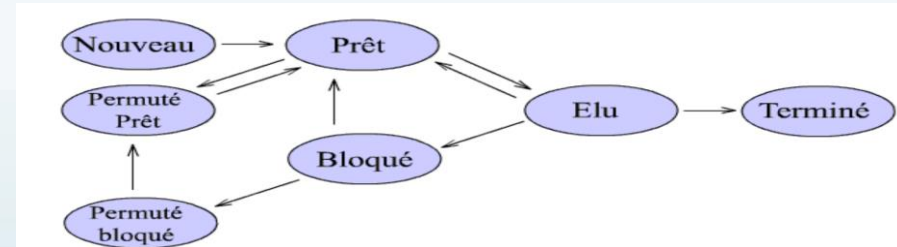
L'ordonnancement long terme

Ou par travaux job scheduling décide des processus que le système peut mener en parallèle.



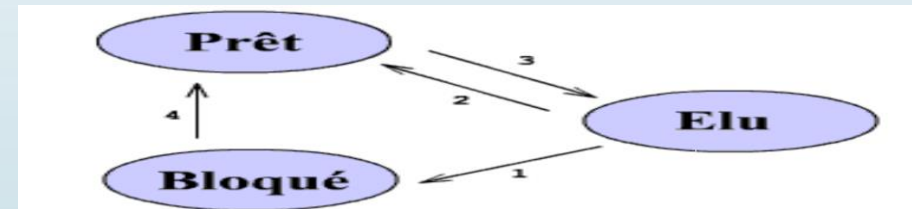
L'ordonnancement à moyen terme

assuré par l'ordonnanceur de mémoire aussi appelé permutateur ou swapper. Son rôle est de permuter les processus placés en mémoire



L'ordonnanceur à court terme

Ou dispatcher, répartiteur ou ordonnanceur du processeur choisit à quel processus sera alloué le processeur et pour quel laps de temps.



Type
Ordonnancement

Non Préemptif

1

L'ordonnanceur attend que le processus actuel se termine ou se bloque.

Exemple : Un serveur qui attend que chaque client termine sa commande.

2

Préemptif

L'ordonnanceur peut interrompre un processus pour en traiter d'autres

Exemple : Un serveur qui aide plusieurs clients à la fois.

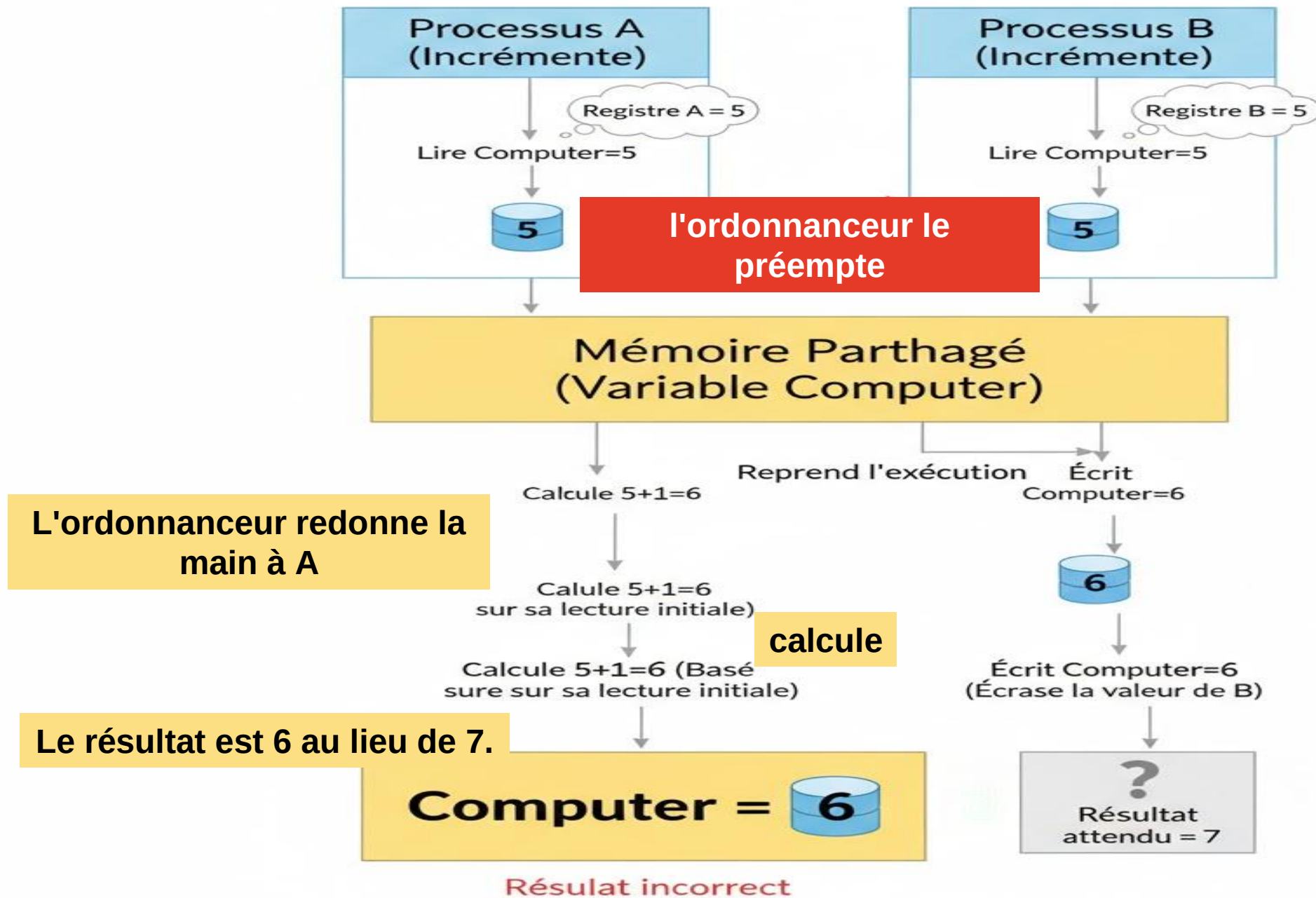
Communication et synchronisation des processus

Scénario : Imaginez deux processus gérés par l'ordonnanceur qui essaient tous deux de mettre à jour une **ressource partagée** (comme une variable en mémoire ou un fichier) **sans coordination**.

Problème

L'ordonnanceur peut interrompre un processus *entre* ses opérations, ce qui peut laisser la ressource dans un état incohérent. C'est la **condition de course** (**Race Condition**).

Exemple : Processus A, lit Compteur=5, l'ordonnanceur le préempte. Processus B, lit Compteur=5, incrémente à 6, et écrit. L'ordonnanceur redonne la main à A, qui ignore l'action de B, calcule $5+1=6$ et écrit. Le résultat est 6 au lieu de 7.



Communication et synchronisation des processus

Il existe plusieurs techniques de communication et synchronisation entre les processus:

- Condition de concurrence
- La section critique
- Les sémaphores
- Le sommeil et l'activation
- Les moniteurs
- L'échange de message
- Les barrières

Le Mutex (Exclusion Mutuelle)

Le **Mutex** (de l'anglais *Mutual Exclusion*) est l'outil le plus simple et le plus couramment utilisé pour résoudre les conditions de concurrence. Il agit comme une **clé unique** pour accéder à une ressource critique.

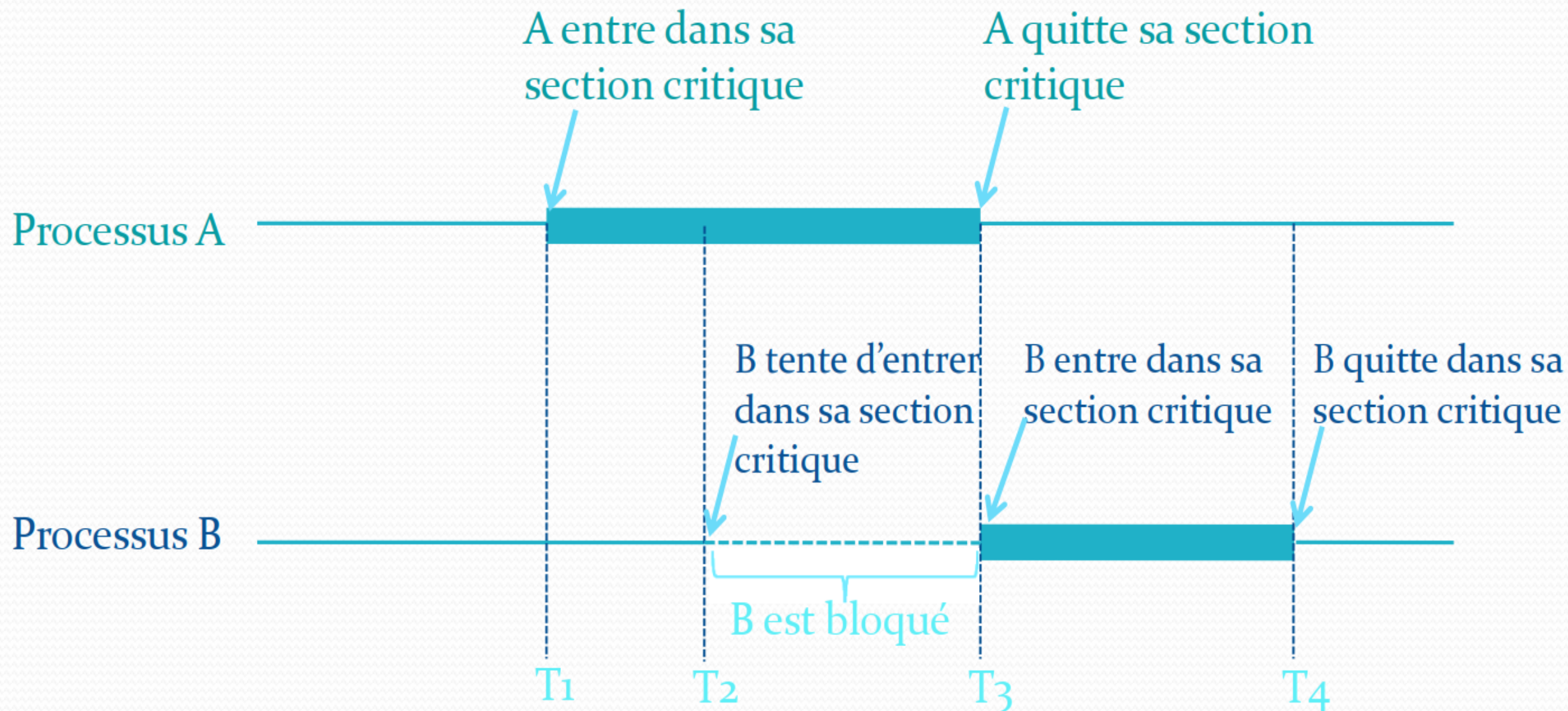
Concept

Garantir qu'**un seul processus à la fois** puisse exécuter la **section critique** (le code qui manipule la ressource partagée).

Mécanisme

- ✓ Un processus doit prendre le **verrou (lock)** pour entrer dans la section critique.
- ✓ S'il est déjà pris, le processus est **bloqué** (mis en attente).
- ✓ Une fois le travail **terminé**, le processus doit libérer le **verrou (unlock)**.

Section critique



Sémaphores

Le **Sémaphore** est un outil plus général. Il peut être utilisé pour l'exclusion mutuelle, mais aussi pour gérer l'accès à un nombre limité de ressources, ou pour **signaler** qu'un événement s'est produit.

Concept

Contrôler l'accès à une ressource ou coordonner l'exécution.

Mécanisme

- ✓ $P()$ ou $wait()$: **Décrémenter** le compteur. Si le résultat est négatif, le processus est bloqué.
- ✓ $V()$ ou $signal()$: **Incrémenter** le compteur. Si le résultat devient positif ou nul, un processus bloqué est réveillé.

Type de Sémaphore

Binaire: similaire à l'exclusion mutuelle

Compteur : Utilisé pour limiter l'accès à N ressources identiques.

Programme vs Processus

Programme : Le code statique (les instructions) stocké sur le disque dur. C'est la recette.

Processus : Une **instance** d'un programme en cours d'exécution. C'est la recette en cours de préparation dans la cuisine, avec ses ingrédients (données), son espace de travail (mémoire), et son chef (le CPU).

Création

Fork: Crée un **doublon exact** du processus appelant (le parent), y compris son code, ses données, et son état

retourne **0** au **processus enfant**.

retourne le **PID** de l'enfant au **processus parent**.

Exec: Est généralement appelé immédiatement après `fork()` par le processus enfant. Il remplace l'espace mémoire et le code du processus enfant par un nouveau programme

Sous Windows vous pouvez faire la commande : `start-process`

```
Start-Process "notepad.exe"
```

Exécution d'un processus

Exécution en Série (ou Séquentielle)

Les processus s'exécutent **l'un après l'autre**. Le Processus B ne commence que lorsque le Processus A est **terminé**.

Synchronisation : La fonction **wait()** est utilisée par le parent pour **bloquer** son exécution jusqu'à ce que son enfant ait fini.

Exemple: Compiler d'abord un fichier source, puis exécuter le programme obtenu.

Exemple sous Linux:

```
echo "Début de la Tâche 1." ; sleep  
5 ; echo "Fin de la Tâche 2."
```

```
PS C:\Users\NBH> Write-Host "--- Étape 1 : DÉMARRAGE SÉQUENTIEL ---" ; `
>> Write-Host "  -> Lancement du Bloc-notes et ATTENTE. Fermez-le pour continuer..." ; `
>> Start-Process "notepad.exe" -Wait ; `
>> Write-Host "--- Étape 2 : L'ATTENTE est TERMINÉE ---" ; `
>> Write-Host "  -> Le Bloc-notes a été fermé. L'exécution se poursuit."
--- Étape 1 : DÉMARRAGE SÉQUENTIEL ---
  -> Lancement du Bloc-notes et ATTENTE. Fermez-le pour continuer...
--- Étape 2 : L'ATTENTE est TERMINÉE ---
  -> Le Bloc-notes a été fermé. L'exécution se poursuit.
```

Exécution d'un processus

Exécution en Parallèle (ou Concurrente)

Les processus s'exécutent **simultanément** (si plusieurs cœurs CPU sont disponibles) ou **concurrentiellement** (grâce à l'ordonnanceur).

Synchronisation : Les mécanismes que vous avez vus (Mutex/Sémaphore) sont cruciaux pour gérer les accès aux données .

Exemple: Ouvrir un tableur, un navigateur web et un lecteur de musique en même temps.

```
PS C:\Users\NBH> Start-Process calc.exe ; Start-Process explorer.exe
PS C:\Users\NBH>
```

```
C:\Users\NBH>start calc.exe & explorer.exe
```

```
root@manale-VMware-Virtual-Platform:~# sleep 5 & sleep 3 & echo "C'est fini!"
[1] 8251
[2] 8252
C'est fini!
```

&: nous permet de lancer la commande en arrière plan de ce fait le terminal sera libre pour la commande suivante.

Exécution d'un processus

Exécution Pipe

Un type spécial d'exécution en série où la **sortie** d'un processus est directement connectée à l'**entrée** du processus suivant. C'est un tube de communication unidirectionnel (FIFO - *First In, First Out*).

Synchronisation : Utilisation de la communication inter-processus (IPC).

Exemple: Compiler d'abord un fichier source, puis exécuter le programme obtenu.

```
PS C:\Users\NBH> Get-Process | Where-Object {$_.CPU -gt 100} | select-Object -First 3
```

Handles	NPM(K)	PM(K)	WS(K)	CPU(s)	Id	SI	ProcessName
2528	56	48452	77880	573,42	7056	1	AvastUI
428	42	322040	380716	429,59	1576	1	chrome
2193	86	208892	265312	644,28	13916	1	chrome

```
root@manale-VMware-Virtual-Platform:~# ls -l /etc | grep "conf"
-rw-r--r-- 1 root          root      3444 juil.  5  2023 adduser.conf
-rw-r--r-- 1 root          root      433 avril  8  2024 apg.conf
```

Ordonnancement des Processus

L'ordonnancement de Linux est fondé sur les threads et non sur les processus. Les threads sont implémentés dans le noyau. Linux distingue trois classes de threads pour l'ordonnancement

1. Les FIFO temps réel
2. Les tourniquets temps réel
3. Les threads en temps partagé

FCFS First-come First-served

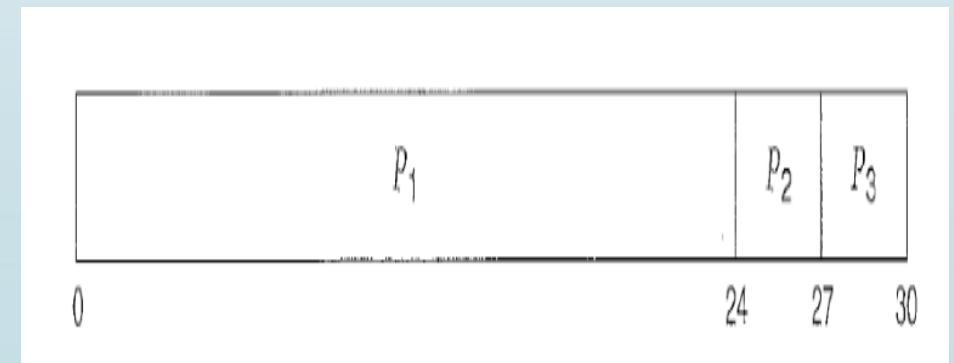
Les jobs attendent dans une file. Le premier arrivé est admis immédiatement et s'exécute tant qu'il n'est pas bloqué ou terminé. Lorsqu'il se bloque, le processus suivant commence à s'exécuter et le processus bloqué va se mettre au bout de la file d'attente. C'est typiquement un algorithme non préemptif.

- **Avantage:** algorithme simple et équitable.
- **Inconvénient:** le processus qui utilise d'avantage de temps processeur est favorisé par rapport à ceux qui font beaucoup d'appels E/S.

Exemple:

Considérez l'ensemble de processus suivant qui arrivent au temps 0ms, avec le temps d'exécution donné en millisecondes :

Processus	Temps d'exécution
P1	24
P2	3
P3	3



Shorted job first

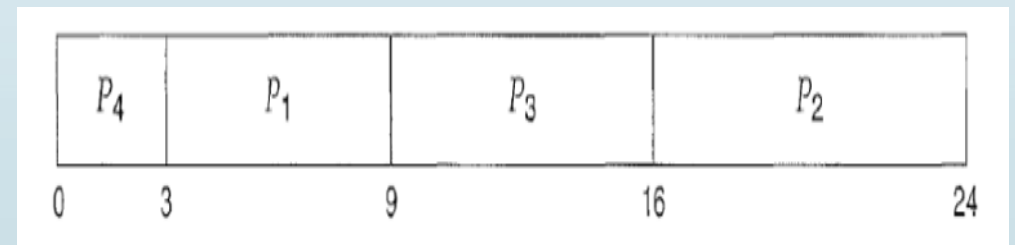
Le job le plus court d'abord sera élu, le processus dont on suppose que le traitement sera le plus court. Si quatre jobs ont des durées d'exécution a, b, c et d ; le premier se termine à l'instant a, le second à l'instant a+b et ainsi de suite.

Le premier job exécuté contribue donc beaucoup plus que les autres à la durée moyenne. Il est donc normal d'exécuter le plus court en premier lieu. (C'est ce qui se passe quand à la caisse d'une grande surface les clients laissent passer devant quelqu'un qui n'a qu'un article).

Exemple: Considérez l'ensemble de processus suivant qui arrivent au temps 0ms, avec le temps d'exécution donné en millisecondes:

Processus	Temps d'exécution
P1	6
P2	8
P3	7
P4	3

Diagramme de Gantt



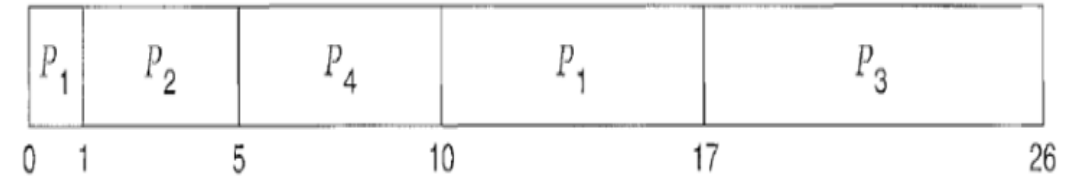
Shorted remmaining time

C'est la version préemptive de l'algorithme précédent. (Cette fois le client qui n'a qu'un article vient carrément interrompre la caissière.)

Exemple: Considérez l'ensemble de processus suivant qui arrivent au temps 0ms, avec le temps d'exécution donné en millisecondes

Processus	Temps d'arrivée (T_0)	Temps d'exécution (ΔT)
P1	0	8
P2	1	4
P3	2	9
P4	3	5

Diagramme de Gantt



Round robin

Chaque processus reçoit tour à tour un intervalle de temps appelé quantum. Au terme de ce quantum ou, si le processus s'achève ou se bloque avant cet instant, l'ordonnanceur attribue directement le processeur au processus suivant. L'algorithme est simple et équitable. C'est généralement cet ordonnancement circulaire qui est utilisé dans les systèmes à temps partagé. La valeur du quantum est couramment fixée aux alentours de 20 à 50 ms.

Des quanta plus courts provoqueraient trop de commutations de processus et la proportion du temps consacré à ces changements de contexte deviendrait trop importante.

Si par contre, on optait pour des quanta plus longs, ce seraient les temps de réponse aux processus interactifs qui en pâtiraient.

Round robin

Supposons que nous avons trois processus P1, P2, et P3 et Quantum de temps : 2 unités:

- P1 a besoin de 5 unités de temps.
- P2 a besoin de 3 unités de temps.
- P3 a besoin de 4 unités de temps.

1. Déterminer le Diagramme de Gantt.
2. Déterminer le temps moyen d'exécution, et temps d'attente.

Round robin

Supposons que nous avons trois processus P1, P2, et P3 et Quantum de temps : 2 unités:

- P1 a besoin de 5 unités de temps.
- P2 a besoin de 3 unités de temps.
- P3 a besoin de 4 unités de temps.

P1	P2	P3	P1	P2	P3	P1
----	----	----	----	----	----	----

Temps d'attente pour chaque processus :

- P1 : Temps d'attente = Temps de fin - Temps d'exécution
 $12 - 5 = 7$ unités.
- P2 : Temps d'attente = $9 - 3 = 6$ unités.
- P3 : Temps d'attente = $11 - 4 = 7$ unités.

Temps moyen d'attente:

$$(7+6+7)/3=6.67 \text{ unités de temps}$$

L'ordonnancement avec priorité

Une valeur de priorité est assignée à chaque processus. Elle peut être fonction d'un ordre de préséance entre utilisateurs ou fonction des caractéristiques des processus. Un processus temps réel sera par exemple prioritaire par rapport à une tâche de fond. La priorité peut varier dynamiquement.

Exemple : Pour ne pas encombrer la mémoire avec des processus qui passent le plus clair de leur temps à attendre des entrées/sorties, on leur accorde une priorité d'autant plus grande qu'ils ne consomment qu'une petite fraction de leur quantum. Supposons que la valeur du quantum est fixée à 50 ms. Un processus qui n'utilise que 1 ms avant d'être bloqué aurait droit à une priorité de $50 \text{ ms} / 1 \text{ ms} = 50$ tandis qu'un processus qui se bloque au bout de 25 ms a droit à une priorité de $50/25=2$ et les processus qui consomment tout le quantum ont une priorité de 1.

L'ordonnancement avec priorité

Soit P1, P2, P3:

- P1 a besoin de 5 unités de temps, avec une priorité de 2.
- P2 a besoin de 3 unités de temps, avec une priorité de 1.
- P3 a besoin de 4 unités de temps, avec une priorité de 3

Donnez l'ordre de d'exécution des processus. Et le temps moyen et le temps d'attente des processus.



Temps d'attente = Temps de début - Temps d'arrivée

Temps attente pour:

$$P1 = 3 - 0 = 3$$

$$P2 = 0 - 0 = 0$$

$$P3 = 8 - 0 = 8$$

$$\text{Temps moyen: } (3 + 8 + 0) / 3 = 3,67$$



Module

Système d'Exploitation et Programmation Système

Chapitre 3

Processus & Threads (1/2)

Pr. MANALE Boughanja

m.boughanja@ump.ac.ma